PORTER: A Plan-Committed Router for Multi-Core Distributed Quantum Computers

Fuchao Sheng, Yongjian Xie, Yongming Li, Sanjiang Li *

***Abstract*—A distributed quantum computer runs one circuit across several small cores joined by links that distribute entanglement on demand. Under the usual cost model the compiler's objective is to spend as few Einstein–Podolsky–Rosen (EPR) pairs as possible on gates that straddle cores. Existing routers choose among inter-core actions while simultaneously staging them, so an action's apparent value depends on the layout the compiler happens to occupy. We present PORTER, a compiler that instead scores a complete inter-core *plan* — which primitive fires, which gates it resolves, and which readiness conditions it needs — before any preparation SWAP for it exists, commits to one, and only then prepares it. PORTER supports all three known inter-core primitives, including cat-entangler packets that amortise one pair over a run of remote gates sharing a control, and adds capacity-aware relief, a deterministic seed-free initial layout, and a per-circuit choice of teleport-lookahead horizon. On MQT-Bench suites at 25, 64 and 80 logical qubits PORTER uses 59.6%, 41.6% and 68.2% fewer EPR pairs in geometric mean than published TELESABRE at a matched attempt budget, and completes all twenty-four instances where TELESABRE completes seventeen. Against DSABRE, concurrent work re-run here from a pinned revision, PORTER is competitive, and which compiler leads depends on the operating point: it uses 7% more pooled EPR at $0.58\times$ the compile time under each compiler's own protocol, and leads once the initial layout is controlled and at the largest scales evaluated. Controlled ablations attribute these results to individual mechanisms and report two negative findings that bound them. Every PORTER route is certified by a separately implemented replay verifier; the implementation, verifier, harness and raw results are public.**

***Index Terms*—distributed quantum computing, quantum circuit compilation, multi-core quantum architectures, gate teleportation, cat-entangler packets, plan-oriented routing, entanglement cost**

I. INTRODUCTION

A distributed quantum computer (DQC) runs one circuit across several small quantum processing units — *cores* — joined by links that distribute entanglement on demand [1], [2]. What such a machine is short of is not gates but entanglement. Any two-qubit gate whose operands sit in different cores has to be bought with Einstein–Podolsky–Rosen (EPR) pairs, and on current hardware distributing a pair is far slower and far less reliable than any operation performed inside a core [3], [4]. How few pairs a compiler spends is therefore the primary optimisation metric under the cost model used throughout this paper (Section II-B), and the one we minimise.

F. Sheng, Y. Xie, and Y. Li are with the School of Mathematics and Statistics, Shaanxi Normal University, Xi'an, China; S. Li is with the Centre for Quantum Software and Information, University of Technology Sydney, Australia (e-mail: sanjiang.li@uts.edu.au).

*ORCID: 0000-0002-3332-2546.

There is more than one way to spend a pair. A *state* can be teleported (TELEDATA), relocating a logical qubit so every later gate it meets in its new core is free [5]. A *gate* can be teleported (TELEGATE) through a cat-entangler, executing one remote CX with neither operand moving [6], [7]. Or one cat-entangled resource can be held open across a run of remote CX gates sharing a control, so the run costs one pair instead of one each — a *packet*, in the spirit of AutoComm's gate-sequence amortisation [8]. Which of the three is cheapest depends on the circuit structure where it is applied, not on the primitive, so none of them dominates.

The choice must be made before its price is known. Each alternative becomes executable only after its own preparation — the right operand beside the right communication port, that port free, the destination holding a spare slot — and preparation is work the compiler will not perform unless it selects that alternative. Scoring an action together with the preparation it currently needs therefore ranks candidates by the layout the compiler happens to be sitting in rather than by the progress they buy, and staging a losing candidate still perturbs the layout, so effort is spent preparing actions never taken.

PORTER — a *Plan-Oriented Router for Teleportation-Enabled aRchitectures* — keeps the two questions apart. Its unit of choice is a *plan*: a complete specification of one inter-core action that fixes what the action is and what conditions it needs, but deliberately omits the SWAP sequence that would establish them. Whole plans are compared on the gate progress they buy; one is committed; only then are preparation SWAPs generated, for that plan alone. Around this loop PORTER adds a two-tier capacity-relief mechanism that manufactures free slots when every candidate is capacity-blocked, and a deterministic, seed-free initial layout replacing the seeded best-of-N search its predecessors run.

Results. We evaluate on MQT-Bench [9] suites at 25, 64 and 80 logical qubits, on a 100–360-qubit scale-up series, and on two networks of IBM heavy-hex cores — forty-eight circuit–architecture instances. Against TELESABRE [10], the published state of the art for this cost model, PORTER uses substantially fewer EPR pairs: over the instances both complete it lowers geometric-mean consumption by 59.6%, 41.6% and 68.2% on the three suites at the same three-attempt budget, and it completes every instance, including the seven — among them an 80-qubit, 23,381-CX circuit — on which TELESABRE does not converge in three seeds.

The second comparison is less one-sided and we report it in full. DSABRE [11] is concurrent work on the same problem whose router was substantially revised while this

paper was in preparation, so we re-ran it from source rather than quoting its tables. Against that version PORTER is *competitive rather than dominant*. Under each compiler's own protocol PORTER costs $1.07\times$ DSABRE's pooled geometric-mean EPR at $0.58\times$ its compile time; restoring a comparable pass budget reaches $1.05\times$ at $0.74\times$. Routed from identical externally supplied layouts at matched pass budgets PORTER moves *ahead* across forty-one instances (126.6 against 134.2), and the same control takes both heavy-hex core graphs. On the 100–360-qubit series it *leads* outright by $0.72\times$ — a reversal traced in Section IV-F1 to a single mistuned constant, the lookahead horizon, which no one value serves across the benchmark. The two compilers share four load-bearing design decisions, set out in Section VII so that neither is credited with the other's.

What the mechanisms are worth. Because PORTER and DSABRE are close relatives, a head-to-head number says little about which design decision earns its place, so we ablate PORTER's in isolation and report the outcome as measured. Capacity relief is what makes a deterministic-greedy design complete in practice: without it four of six 80-qubit instances fail to route. The initial layout is what separates the two compilers below roughly a hundred physical qubits, and the lookahead horizon is what separates them above it. Two answers are negative, and we state them plainly. Plan *commitment* — the rule that a selected plan is not reopened until it is ready — reaches the same EPR as re-scoring the whole front every iteration, within 0.6% and on the same number of routing moves, so what it buys is compile time rather than better routes; note that this ablates the commitment rule and not the plan abstraction it sits on, which no experiment here removes (Section VI). And the packet primitive, which exists to amortise a pair across a run of gates, is outperformed at 80 qubits by removing gate teleportation altogether, by 32.8% — an interaction between the action vocabulary and the plan scorer rather than a verdict on amortisation.

Problem. Given a circuit DAG on n logical qubits, a machine of K cores with intra-core coupling graphs, inter-core links and designated communication ports, and an initial layout, emit a legal operation stream realising every gate while minimising the number of EPR pairs consumed. Section II-B states the cost model this objective assumes and what it excludes.

Contributions.

(i) *Algorithmic.* A plan-oriented remote stage that scores a complete inter-core action on the gate progress it buys, before any preparation SWAP for it exists, and commits to one before generating preparation (Section III-D).

(ii) *Algorithmic.* A capacity discipline that treats free slots as a legality condition and manufactures them by a two-tier relief search when every candidate is blocked (Section III-G).

(iii) *Algorithmic.* A deterministic, seed-free initial layout that replaces the seeded best-of-N search its predecessors run (Section III-A).

(iv) *Empirical.* The observation that the teleport-lookahead horizon — a parameter every SABRE-descended router fixes by constant — has no good constant, its best value spanning a factor of thirty-two across our benchmark, and can instead be chosen per circuit by routing a small pruned candidate set (Section IV-F1).

(v) *Empirical.* A controlled study over forty-eight circuit–architecture instances under native, shared-layout and cross-layout protocols, with mechanism ablations that include two negative results (Sections V–VI).

(vi) *Infrastructural.* A separately implemented replay verifier that re-derives gate coverage, dependency order and physical legality from the emitted operation stream alone, tested by fault injection, and a released harness reproducing every table.

No single mechanism accounts for the results, and Section IV-A poses the four questions the evaluation answers. Each mechanism is tied to the experiment that assesses it: plan commitment to the hop-wise ablation, capacity relief to the completion ablation, the initial layout to the shared- and cross-layout controls, horizon selection to the scale-up series and the star-Multiplier livelock, packets to the action-vocabulary study, and the verifier to fault injection.

The PORTER implementation, the replay verifier, the full cross-router comparison harness, and the raw per-circuit results are publicly available at https://github.com/ebony72/porter.

II. Model, Primitives, and Problem Formulation

A. Quantum Circuits and Routing

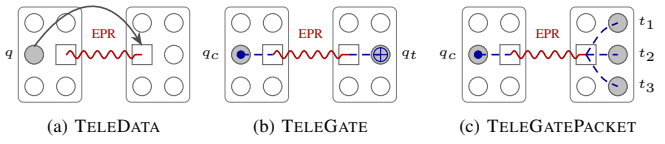
A circuit acts on n logical qubits through a sequence of gates; we work in the restricted basis $\{\texttt{id}, \texttt{rz}, \texttt{sx}, \texttt{x}, \texttt{cx}\}$ of MQT-Bench's Qiskit-opt3-transpiled circuits [9], i.e. arbitrary one-qubit gates plus the two-qubit CX. A circuit is a directed acyclic graph (DAG): for every logical qubit, consecutive gates acting on it are joined by a dependency edge. The *front* is the set of gates with no unexecuted predecessor; only front gates can execute or be routed towards execution.

Physical qubits are partitioned into *cores*; each core has an intra-core coupling graph, and pairs of cores may be joined by an inter-core link whose two endpoints are designated *communication ports*. A *layout* maps every logical qubit to a physical position. A two-qubit gate is *local* if both operands share a core and *remote* otherwise; routing a local gate that is not yet adjacent requires intra-core SWAPs, and routing a remote gate requires an inter-core communication primitive. We write $\kappa_\ell(q)$ for the core holding q under layout ℓ, $d_{\mathrm{L}}(c, p, p')$ for intra-core distance and $d_{\mathrm{C}}(c, c')$ for core-graph distance.

B. Objective, Scope, and Assumptions

Objective. Given a circuit DAG C on n logical qubits, a machine $\mathcal{A}$ of K cores with intra-core coupling graphs, inter-core links and designated communication ports, and an initial layout ℓ_0, produce a legal operation stream that realises every gate of C while minimising the number of EPR pairs consumed. Intra-core SWAP count, circuit depth and compile time are recorded but are not part of the objective and are never traded against EPR pairs by any rule in this paper.

Cost model. One EPR pair is consumed per TeleData, per TeleGate, and per TeleGatePacket — one per packet,



(a) TELEDATA (b) TELEGATE (c) TELEGATEPACKET

Fig. 1. The three communication primitives; each consumes one EPR pair between the two communication ports (squares). (a) TELEDATA moves qubit q (which must sit adjacent to its free port) onto the free destination port. (b) TELEGATE executes one remote CX through a cat-entangler [7] (Appendix E): the target-side port acts as a proxy for the control q_c; neither qubit moves. (c) TELEGATEPACKET brackets an ordered run of remote CX gates sharing the control q_c between one cat-entangler/disentangler pair, amortising a single EPR pair over the whole run; each target is routed next to the port in turn while the ports stay reserved.

however many remote CX gates the packet amortises — and a pair is charged the same regardless of which link carries it. A multi-hop relocation is a sequence of single-hop TELEDATA actions and is charged once per hop. This is the model TELESABRE [10] and DSABRE [11] use, which is what makes the three compilers comparable in a single unit.

Excluded. The model abstracts away everything about how a pair is produced and consumed: entanglement-generation latency and fidelity, purification, link contention and concurrency between links, classical communication and measurement latency, decoherence during a reserved cat resource, and gate error rates. Communication ports are ordinary physical qubits that are occupied for the duration of an action and free otherwise, and no core is assumed to have dedicated communication memory. Every CX is treated as directed, with direction reversal assumed compiled away by the single-qubit basis.

What this scopes. EPR consumption is the primary metric *under this model*. On hardware where link contention or entanglement fidelity dominates, a compiler minimising pair count need not minimise runtime or error, and a policy that serialises many actions across one link may be worse than its pair count suggests. We do not evaluate that regime. Section VI-A restates the consequences for the conclusions drawn here.

C. Communication Primitives

Three inter-core primitives are in play, sketched in Figure 1; each consumes exactly one EPR pair.

State teleportation (TELEDATA). Consumes one EPR pair to move a logical qubit from a communication port in one core to a free port in another, after which every later gate that qubit meets in its new core is local. Its value is therefore *latent*: it is paid once and repaid over however many future gates the relocation makes local.

Gate teleportation (TELEGATE). Executes a single remote CX with neither operand moving. One EPR pair entangles the two communication ports; a *cat-entangler* — $\mathrm{CX}(q_c, e_A)$, measurement of e_A, and a classically controlled X on e_B — leaves (q_c, e_B) in the cat state $\alpha|00\rangle + \beta|11\rangle$, so every CX controlled by the remote port e_B acts as if controlled by q_c. A *cat-disentangler* — Hadamard on e_B, measurement, a classically controlled Z on q_c — restores q_c and frees both ports (Appendix E). Its value is entirely *direct*: exactly one gate, and nothing is left behind.

Same-control packets (TELEGATEPACKET). When several remote CX gates share the same control and their targets lie in one core, the run can be bracketed between a *single* cat-entangler and disentangler: the ports stay reserved, each target is routed beside e_B in turn, and the whole run costs one EPR pair instead of one each. This is the amortisation idea of AutoComm [8], specialised to the shared-control case and applied inside a routing loop rather than as a pre-pass.

The three differ in exactly one respect that matters for scoring: how many gates one pair is spread over. TELEDATA amortises over an unknown future, TELEGATE over exactly one gate, and TELEGATEPACKET over a run whose length the compiler must discover. Section III-E makes that difference explicit as a direct/latent/lookahead decomposition.

SABRE. SABRE [12] routes a single-core circuit by repeatedly selecting the SWAP that minimises a front-layer distance term plus a weighted extended-set lookahead term, both expressed as Δ (old distance minus new distance) on gates sharing a qubit with the candidate SWAP. PORTER inherits this scoring form for its intra-core stage (Section III-C) and extends the same decomposition to its inter-core plan scorer (Section III-E).

III. THE PORTER COMPILER

Given a circuit and an initial layout ℓ_0 (which PORTER builds itself, Section III-A), PORTER repeats the following loop (Algorithm 1) until every gate has been emitted or a configurable iteration budget is exhausted. Algorithm 1 charts one iteration; its shaded region is the plan-first remote stage that distinguishes PORTER from compilers deciding at single-action granularity; Appendix F walks through it on a concrete front where it changes the outcome.

A. Initial Layout

PORTER builds its starting layout ℓ_0 with one deterministic, seed-free heuristic — no external partitioner, no best-of-N search — and respects the capacity invariant the routing loop depends on: every core keeps at least $f_{\min}$=2 free positions, so from the first iteration any core can serve as a TELEDATA destination and as a relief reservoir.

Temporal locality-greedy. Three deterministic steps. *(i) Weighting*: each CX gate $g = (q_i, q_j)$ contributes $\gamma^{\mathrm{asap}(g)}$ to the interaction weight $w(q_i, q_j)$, where $\mathrm{asap}(g)$ is g's as-soon-as-possible layer in the CX-only DAG and γ=0.99, so early interactions weigh more than late ones while repeated interactions accumulate. *(ii) Core assignment*: logical qubits are visited in the order they first appear as a CX operand and each is placed in the core holding its heaviest already-placed neighbour when that core has headroom above $f_{\min}$, falling back to the core with the most remaining reserved capacity. *(iii) Physical placement*: within each core, logicals are laid onto positions by descending interaction weight onto ascending intra-core centrality, so the most-connected qubits take the most central slots.

Algorithm 1 PORTER `compile`(C, ℓ_0)

$\ell \leftarrow \ell_0$; initialise DAG active/frontier sets from C
while DAG has active gates **do**
  $F \leftarrow$ DAG front (bounded by `dag_front_limit`)
  classify F into singles, direct-local CX, local-CX-needing-SWAP, adjacent-remote CX, non-adjacent-remote CX *(Sec. III-B)*
  if singles or direct-local CX present **then**
    execute them all; remove from DAG; **continue**
  end if
  if local-CX-needing-SWAP present **then**
    generate candidate intra-core SWAPs, apply the best-scoring one (Eq. 1); **continue** *(Sec. III-C)*
  end if
  generate normal remote plans $\mathcal{P}$ and blocked-teleport plans $\mathcal{B}$ *(Sec. III-E)*
  if $\mathcal{P} \neq \emptyset$ **then**
    $p^* \leftarrow$ best ready-or-preparable plan in $\mathcal{P}$
    if some $b \in \mathcal{B}$ has gain $\gg$ gain(p^*) **then**
      $p^* \leftarrow$ best opportunistic capacity-relief plan for b *(Sec. III-G)*
    end if
  else
    BFS from each blocked target core to its nearest reservoir; $p^* \leftarrow$ best tail-relief plan *(Sec. III-G)*
  end if
  if p^* is `None` **then**
    report deadlock; abort
  end if
  apply p^*'s own preparation SWAPs until ready, then execute it
end while
return emitted operations, final layout, stats

The steps are complementary. The temporal discount front-loads the opening layers, co-locating the pairs that must interact soonest; accumulated multiplicity plus first-appearance order tracks chain-structured circuits, where only one disjoint pair is exposed per layer and a balanced-first-layer rule would scatter the chain across cores; and the centrality step keeps hub qubits cheap to reach. The construction costs microseconds and consumes no randomness, so PORTER has no seed to report.

B. Front Classification

Every front gate is classified under the current layout ℓ. 1Q gates are always directly executable; a CX $g = (q_i, q_j)$ with both operands in core c is *direct-local* if $d_{\mathrm{L}}(c, \ell(q_i), \ell(q_j)) = 1$ and *local-needing-SWAP* otherwise; a CX whose operands lie in different cores is *adjacent-remote* if those cores are core-graph neighbours and *non-adjacent-remote* otherwise. Directly executable gates drain first and completely, intra-core SWAPs are exhausted next, and remote-stage decisions are taken only once no front gate is intra-core. The front is size-capped at dag_front_limit=96 gates to bound per-iteration cost. Front gates are pairwise qubit-disjoint, so the front can never exceed the logical-qubit count n; single-qubit gates are not excluded by that bound, however, and on the three QPEexact instances of the scale-up suite the front does reach n (100, 200 and 360) and the cap binds. It is inert where it binds: truncation is by ascending gate index, directly executable gates drain before any routing decision is taken, and the discarded gates therefore reappear on the following iteration. Re-routing all three with the cap removed leaves EPR and SWAP counts unchanged. On every other instance evaluated here the front stays at or below 47 gates and the cap does not bind. Both figures are reproduced by `probe_front_limit.py` in the artifact.

C. Intra-Core SWAP Scoring

For a local-needing-SWAP gate set F_{loc}, PORTER builds candidate SWAP edges from the coupling-graph neighbours of every physical position touched by F_{loc}, simulates each SWAP, and scores it with SABRE's front-plus-lookahead form [12]:

$$J(a) = \bar{d}_{\mathcal{F}}(a) + \lambda_{\mathrm{loc}}\, \bar{d}_{\mathcal{E}}(a), \qquad \lambda_{\mathrm{loc}} = 0.8, \tag{1}$$

where $\bar{d}_{\mathcal{F}}(a)$ is the mean, over the gates $(q_i, q_j) \in F_{\mathrm{loc}}$, of the intra-core distance $d_{\mathrm{L}}\big(c, \ell_a(q_i), \ell_a(q_j)\big)$ under the post-SWAP layout ℓ_a, and $\bar{d}_{\mathcal{E}}(a)$ is the same mean taken over an extended set $\mathcal{E}_{\mathrm{loc}}$ of up to N_{loc}=20 local CX gates collected from the next L_{loc}=12 DAG layers via a virtual frontier simulation (Section IV-C). Ties are broken first by the raw front-layer distance, then, if still tied, by a decision uniformly sampled from a constant-seeded random-number generator, with every such tie-break recorded in `RouterStats` for reproducibility auditing.

D. Plan-First Remote Stage

PORTER manipulates remote work as *plans*. A plan is a complete, committable specification of one inter-core action: which primitive fires, which gate or same-control run it resolves, its direction, and consequently its *readiness conditions* — which logical endpoint must sit adjacent to which communication port, and which ports must be free. What it deliberately leaves unspecified is the SWAP sequence that establishes those conditions. Alternatives are compared at the level of *what* remote work to do next, each scored as if its preconditions were met; only the committed plan is handed to the machinery that works out *how*.

Formally, a plan is a tuple

$$p \;=\; \big(\pi,\; G(p),\; \mathrm{dir},\; c_{\mathrm{src}},\; c_{\mathrm{dst}},\; \Pi(p),\; \mathrm{ready}(p, \ell)\big),$$

with $\pi \in \{\textsc{TeleData}, \textsc{TeleGate}, \textsc{TeleGatePacket}\}$ the primitive, $G(p)$ the front or collected gates it resolves, dir its orientation, c_{src} and c_{dst} the cores involved, $\Pi(p)$ the communication ports it reserves, and $\mathrm{ready}(p, \ell)$ the predicate on layouts that holds when p can fire. Three quantities are kept distinct: the plan's *utility*, evaluated under the hypothesis $\mathrm{ready}(p, \ell)$ regardless of whether it currently holds (Section III-E); its *readiness cost*, used only to break ties; and the *preparation policy* that chooses local SWAPs once p is committed. Two design decisions follow and are what distinguish the remote stage: utility never includes readiness

cost, and a committed plan is not reopened until ready holds. Section VII places both against the baselines' choices; Section V-C measures the second.

When the front holds only remote CX gates, PORTER generates two disjoint candidate sets. **Normal plans** $\mathcal{P}$: for every adjacent-remote CX, a TELEGATE; a TELEGATEPACKET if a same-control run of ≥ 2 remote CX gates can be collected by forward DAG simulation (Section III-F); and, for either endpoint, a TELEDATA plan into the partner's core if that core has at least $f_{\min}$=2 free positions. For non-adjacent-remote CX gates, only distance-reducing TELEDATA hops with sufficient destination capacity. **Blocked plans** $\mathcal{B}$: the same TELEDATA candidates for destinations that currently fail the $f_{\min}$ test. These are never executed; they exist to detect when no normal plan exists because every destination is saturated, or when one is available but a blocked plan promises much more if capacity were relieved first.

Every plan is scored (Section III-E) *before* any preparation SWAP for it exists. PORTER then selects the best plan in $\mathcal{P}$ that is ready or admits an improving preparation SWAP; compares its gain against the best of $\mathcal{B}$ and, if the blocked plan leads by more than m_{assist}=5.0, runs opportunistic capacity relief for that saturated core instead (Section III-G); and, if $\mathcal{P}$ is empty, runs the same breadth-first relief search directly on $\mathcal{B}$'s destinations, shortest hop count first. Once committed, it generates and applies *only that plan's* preparation SWAPs until the plan is ready, no improving SWAP remains, or the full decision state — plan identity, layout, RNG state — repeats, which raises rather than looping.

Preparation SWAPs. A committed plan's readiness requirements are exactly two *(core, neighbour-core, endpoint)* triples (source and destination side for TELEDATA; control and target side for TELEGATE/TELEGATEPACKET, whose target-side routing is deferred to execution time, Section III-F). For each unmet requirement, candidate preparation SWAPs are restricted to edges incident to either the plan's logical endpoint or a nearest free physical position tied for minimum distance to the relevant communication port (only while that port remains occupied); each candidate is scored by the post-SWAP value of the same readiness-cost function used to rank blocked plans (Section III-E), and at most one improving SWAP per core side is applied per pass.

E. Plan Scoring: Direct, Latent, and Lookahead Gain

Every plan scores $\text{total} = \text{direct} + \text{lookahead}$, computed per plan kind but sharing one Δ-form convention: for a CX gate $g = (q_i, q_j)$ and a candidate relocation, $\Delta(g) = d_{\text{C}}(\kappa(q_i), \kappa(q_j)) - d_{\text{C}}(\kappa'(q_i), \kappa'(q_j))$, the reduction in core-graph distance between g's two cores.

A **TELEGATE** contributes a constant direct gain of 1 — one remote CX resolved — and no lookahead. A **TELEGATEPACKET** contributes a direct gain equal to the number of remote CX gates in the collected packet; every gate in an accepted packet is current rather than speculative work, since the whole packet executes under one cat-entangler resource once started. A **TELEDATA** plan decomposes into three tiers, all computed by simulating the DAG frontier under the hypothetical post-teleport core assignment: *direct* — front-layer CX gates already involving the moving qubit, plus the seed gate; *latent-direct* — CX gates on the moving qubit that surface while the simulator can still progress using only 1Q and now-local gates, which are current pressure rather than prediction because the candidate teleport fixes their core distance the moment they appear; and *lookahead* — gates on the moving qubit found in up to L_{tp} further simulated layers, a gate in layer k entering with weight $w_k = \max(w_{\min}, w_0\gamma^{k-1})$, w_0=0.9, γ=0.9, $w_{\min}$=0.5. Thus

$$\text{direct} = \sum_{g \in \text{direct} \cup \text{latent}} \Delta(g), \quad \text{lookahead} = \sum_{(g,k) \in \text{la}} w_k \, \Delta(g), \tag{2}$$

with latent-direct gates entering at full weight alongside direct ones. A capacity-risk term $1/\max(1, f_{\text{dst}}-1)$ is tracked alongside the score for tie-breaking, but never added to it.

This is the point at which PORTER differs most sharply from DSABRE: preparation cost appears nowhere in total. Plans are ranked on gain, and readiness cost enters only as the third tie-break in the selection key, after gain and plan kind. The horizon L_{tp} is counted in *layers*, and the collection admits at most one gate on the moving qubit per layer, because a qubit has at most one zero-predecessor gate at any time. A gate cap N_{tp} therefore cannot bind whenever $N_{\text{tp}} \geq L_{\text{tp}}$, so L_{tp} alone governs how far the estimate looks. It is the one parameter PORTER does not fix by constant; how a value is chosen per circuit is a protocol question and is deferred to Section IV-F1.

F. Same-Control Packet Collection

Same-control CX gates are chained by DAG dependency edges, so they never appear in the front together: a packet cannot be read off the front and must be collected by forward simulation. From a seed adjacent-remote CX $g = (q_c, q_t)$ with control core c_c and target core c_t, PORTER simulates the DAG frontier forward for up to 120 layers, greedily absorbing any front CX sharing control q_c whose target is also in c_t (up to 8 such gates) and any front 1Q gate whose qubit is already in c_t, since those execute freely while the cat resource is live. Collection stops at the first unproductive layer or at the size cap. The window counts simulated *layers*, not collected CXs, because same-control CXs are DAG-chained and surface one at a time; in practice it is a cost guard, not an active knob. Because the simulation admits gates from later generations than the bounded current front, a packet may contain gates a naively front-limited scan could not reach. The collected order is replayed exactly at commit time as one `cat_entangler` (consuming the packet's single pair), one `cat_cx` per member — each target routed to its port by ordinary local SWAPs forbidden from touching the two live ports — and a closing `cat_disentangler`.

G. Capacity Relief

Tail-relief BFS. When every remote plan is capacity-blocked, PORTER runs a breadth-first search from each blocked destination core to the nearest *reservoir* core — one with at least $f_{\min}$ free slots — optionally chained beyond

one hop (`capacity_relief_chain_enabled`). Every logical qubit currently in the BFS path's penultimate core becomes a tail-relief TELEDATA candidate into the reservoir, scored and selected by the same plan machinery, grouped and tried in ascending hop-count order.

Opportunistic relief. Even when a normal plan is available, a severely blocked plan can still win if relieving its target core's capacity would unlock substantially more gain (Section III-D); this differs from tail-relief BFS in firing preemptively, before any plan is actually starved, whenever the margin m_{assist} is exceeded.

H. Progress and the Failure Model

Deterministic greedy routing on a capacity-constrained machine can stall, and the mechanisms above are best read as one answer to that risk. PORTER layers four defences.

(i) Keep capacity from reaching zero. A core with no free physical qubit is terminal under the port-based action model: it can never vacate an occupied communication port with local SWAPs, so no inter-core action can touch it and its qubits are stranded. PORTER legalises the entry layout so no core starts full, and admits a TELEDATA plan into core c as a *normal* plan only while $\text{free}(c) \geq f_{\text{min}}{=}2$. Ordinary routing therefore cannot drive a core into the terminal state; only an externally supplied layout can present one, which is why Appendix B's pytket-dqc source is the only place the compiler reports *infeasibility*.

(ii) Detect starvation rather than thrash. An empty plan set alongside a non-empty $\mathcal{B}$ is an explicit signal that the front is starved *by capacity*, and it names the cores responsible. *(iii) Manufacture capacity where it is missing.* Tail-relief BFS turns that signal into work (Section III-G). *(iv) Bound every loop.* Plan sides whose preparation score is infinite are never selected; during preparation the compiler hashes its full decision state once per round and raises if it repeats; two budgets bound the loops from outside; and when no relief path reaches a reservoir PORTER raises a diagnostic naming the saturated cores.

These defences establish two of the three guarantees a router might offer, and it is worth separating them. *Correctness*: a returned operation stream realises every gate of the input circuit and violates no coupling or port rule. *Termination*: every run ends, either in success or by raising a diagnostic or tripping a guard. *Completion*: that a run ends in success. PORTER provides the first two by construction. It does *not* provide the third, and has no analogue of DSABRE's termination-with-success proof under stated architectural hypotheses (Section VII); its completion record is empirical, and Section V reports it.

A run that does not complete does so in one of four distinguishable ways, and we use these terms consistently. A capacity *deadlock* leaves no legal plan and no relief path, and raises a diagnostic naming the saturated cores. A *livelock* makes local progress indefinitely while retired gates and EPR count stand still, and is stopped by the iteration guard rather than by any diagnostic. A *timeout* exceeds the wall-clock cap. An *infeasible input* is an externally supplied layout that already fills a core, which is reported rather than routed.

A per-iteration cost analysis of the loop is given in Appendix I.

IV. EXPERIMENTAL METHODOLOGY

A. Research Questions

The evaluation answers four questions, each with its own controls, and Section V is organised around them.

RQ1 **End-to-end cost.** Under each compiler's own recommended protocol, how does PORTER's EPR consumption and compile time compare with TELESABRE's and with the evaluated DSABRE revision's?

RQ2 **Routing quality with the layout controlled.** Given the *same* externally supplied initial layout, and at matched routing budgets, how does PORTER's routing loop compare with DSABRE's?

RQ3 **Mechanism attribution.** What does each of PORTER's mechanisms — plan commitment, capacity relief, the packet primitive, the TELEGATE family, the initial layout and horizon selection — contribute on its own?

RQ4 **Robustness.** How do the answers change with circuit width, core count and core-graph shape?

Each results subsection opens with the question it answers, what is held fixed, and the metric; the interpretation of the answers, including the places where they appear to conflict, is collected in Section VI.

B. Terminology

Budget comparisons depend on using these terms consistently, so we fix them here and use no synonyms.

- A **route** (equivalently a *routing pass*) is one invocation of the routing loop of Section III on one circuit from one layout, producing one operation stream.
- A **candidate layout** is one initial mapping a protocol supplies to the router. A protocol's cost is its layout count times its routes per layout.
- A **protocol** is a named row of Table I: a layout source, a route budget and a selection rule. We avoid "attempt", which the baselines use for both meanings.
- A **candidate horizon** is one value of L_{tp} offered to horizon selection; selecting among them costs one route each.
- **Complete compile cost** is the wall-clock time of an entire protocol — layout generation plus every route over every candidate — not of the winning route alone.

For the machine and the router we use: *core*, *communication port*, and *reservoir* (a core holding at least f_{min} free positions); a front CX is *local*, *adjacent-remote* or *non-adjacent-remote*; a plan is *normal* or *blocked*, and *ready*, *preparable* or neither; and a run that does not complete does so by *deadlock*, *livelock*, *timeout* or *infeasible input* (Section III-H).

C. Implementation and Verification

PORTER is implemented in Python and is publicly available together with the verifier and comparison harness. It takes no seed: heuristic ties resolve through a fixed chain of deterministic keys and any residual tie through a constant-initialised generator, and every residual tie-break is counted in

the emitted statistics, so a single routing pass on a fixed circuit and hardware emits a byte-identical operation stream across runs and behaviour preservation across refactorings is certified by hashing it. One caveat applies to the selection protocol of Section IV-F1 and we state it plainly: the speculative horizons are guarded by a wall-clock budget, so on a loaded machine a candidate may be abandoned that would otherwise have won, and the protocol's output is reproducible only up to that budget. Each individual route remains exactly reproducible, and the horizon a run selected is recorded alongside its result. Compile time is kept low by incremental data structures — chiefly a copy-on-write DAG frontier simulator shared by extended-set collection and teleport lookahead, whose snapshots cost $O(|\text{frontier}|)$ rather than $O(|\text{remaining circuit}|)$ — together with cached distance tables, $O(1)$ free-slot accounting, and apply/restore SWAP scoring in place of layout cloning.

Every reported PORTER run is certified by a replay verifier, written separately from the router and sharing no routing, scoring or layout code with it — it consumes only the emitted operation stream and the operation schema, and calls nothing in the routing loop. It treats the compiler as a black box: from the emitted operation stream, the original circuit and the initial layout alone it re-derives gate coverage (each gate exactly once), per-qubit dependency order, coupling-edge legality of every local operation, port rules for every teleport, telegate and packet, and consistency of every reported statistic. The verifier is itself tested by fault injection — ten classes of mutation applied to passing streams, every mutant rejected (Appendix D). All PORTER numbers in this paper are verifier-certified. Baseline routes are *not*: TELESABRE and DSABRE emit their own operation formats, and we report the EPR counts their own harnesses produce, as their published evaluations do.

D. Setup

Architectures. We use the two grid-of-grids topologies DSABRE [11] evaluates on: architecture A, a 2×2 array of 3×3 cores (36 physical qubits, four links), and architecture H, a 2×3 array of 4×4 cores (96 physical, seven links). Sections V-D and V-E add its 5×5-core scalability series and two networks of IBM 27-qubit heavy-hex tiles. All are read from the baselines' own device files, so every compiler sees identical physical qubit ids.

Benchmarks. We use the same MQT-Bench [9] circuit files DSABRE's own evaluation uses, confirmed by exact CX-count match against its published tables: a 25-qubit suite (AE, GHZ, Graphstate, QFT, QNN, Random) on architecture A and a 64-qubit suite (the same six families, plus QPEexact, QAOA and Multiplier) on architecture H, with QNN the deeper 8,126-CX variant. To test at intermediate scale on the same architecture we add an 80-qubit suite of the same nine families. The 80-qubit AE and QNN instances cannot be produced by the stock generator and are reconstructed to match its own construction exactly — in particular QNN is the deep 12,718-CX ZZ-feature-map circuit, not the degenerate 79-CX chain the current generator emits, whose interaction graph is identical to GHZ's. Appendix A details both reconstructions, their byte-level validation, and the CX counts.

Baselines. We compare against TELESABRE [10] and DSABRE [11], the latter in its default `dSABRE_BurstExt` mode with its shipped configuration — which since its most recent revision means *safe mode*: capacity as a legality condition with a per-core reserve, BFS slack relay, and a guaranteed per-gate recovery transaction (Section VII). Its router has changed materially since the version its published tables report, so every DSABRE number here was regenerated from revision `cb5515b` (2026-08-21) rather than quoted, and the harness that produced them is released with PORTER.

Three protocol decisions are worth stating, since each avoids disadvantaging the baseline. We run DSABRE under its own budget convention, letting all three of its recovery budgets derive from the architecture as its own drivers do rather than imposing values of our own — a binding iteration budget would cut its heuristic search short and hand the remainder to its guaranteed drain at a worse EPR count. We do not charge it for pre-routing layout repair: in safe mode `route` brings an externally supplied layout up to its reserve before the clock starts, and those relay teleports land in its EPR counter although they adjust the *starting layout* rather than route anything; PORTER performs the same repair as a free permutation, so we net it out uniformly. And we hand it both families of its own layout candidates rather than picking one (Section IV-E), which can only strengthen it relative to its own published protocol.

TELESABRE's stock reference C binary crashes deterministically on architecture A, in its Hungarian initial-layout pass rather than in routing: a hard-coded capacity threshold leaves some qubits unplaced and their sentinel index is then dereferenced. All TELESABRE numbers here use a minimally patched binary that falls back to the least-loaded core; the patch is in our repository. Every compiler invocation is capped at 900 s wall-clock, uniformly across all three compilers, and a call exceeding it is recorded like any other failure to complete.

E. Initial-Layout Protocols

We evaluate under three initial-layout protocols, designed to separate routing quality from initial-layout quality.

Own-protocol comparison. Each router is first evaluated under its own established best-practice protocol. For the two baselines that is best-of-three, matching the convention already used to compare TELESABRE and DSABRE to each other [11]: TELESABRE runs its built-in Hungarian initial-layout heuristic at config seeds 0–2, and DSABRE runs Qiskit `SabreLayout` on its corners-removed architecture graph at seeds 0–2, each layout routed by SABRE-style forward → backward → forward passes; both report the best EPR count. PORTER instead builds one deterministic temporal locality-greedy layout (Section III-A) and routes it at each of the three candidate lookahead horizons of Section IV-F1, keeping the best — three routing passes over one layout, against nine to eighteen over three to six. It takes no seed and searches no layouts, so every number is exactly reproducible from the circuit file alone.

DSABRE's corner removal reserves low-degree physical qubits as communication slots by excluding them from the SabreLayout coupling map. It ships two variants of the rule and neither dominates on our architectures, so rather than choose we give DSABRE both families and take its best: up to six candidate layouts instead of three. This can only strengthen the baseline relative to its own published protocol, and it is why our DSABRE numbers sit at or below the figures it publishes for the architectures it reports.

Shared-layout comparison. To separate layout quality from routing quality, both DSABRE and PORTER are then run from an identical externally supplied mapping — TELESABRE's own initial layout, and separately `pytket-dqc`'s hypergraph partition (Appendix B) — so the only difference left is the routing loop. **Cross-layout comparison.** Finally each compiler routes the other's own layout, which closes the remaining cell of the design.

F. Operating Protocols

Section III defines one routing pass. Three protocols run that pass more than once and keep the best result by EPR; because they invoke the router repeatedly rather than changing what a single pass does, we describe them here rather than as part of the algorithm. All three are deterministic given the circuit, the hardware and the configuration, and none takes a seed.

1) Lookahead-horizon selection: The teleport-lookahead horizon L_{tp} is the one parameter of Section III-E that no single value serves. In an offline sweep over $\{4, 8, \ldots, 1024\}$ on every instance the best value spans the whole range: 80-qubit QNN wants 4, the 25-qubit suite wants 16, and every 100-, 200- and 360-qubit instance wants 128, beyond which EPR saturates. The gap between the shipped 16 and the per-instance best is 17.2% geometric-mean EPR.

The direction is set by how dense the circuit's interaction graph is. Writing ρ for the mean fraction of the other logical qubits each qubit shares a CX with, a long horizon reports the moving qubit as needed in *every* core at once when $\rho \to 1$, so the gain estimate degenerates into noise and short horizons win; when ρ is small the signal is coherent and long horizons win. Across the benchmark, ρ ranks the best horizon at Spearman -0.765. It is not, however, a usable predictor on its own: in the band $0.42 \le \rho \le 0.53$, where most 64-qubit circuits sit, QAOA wants 8 while AE wants 128, and a threshold rule on ρ recovers only 8.0% of the available 17.2%.

PORTER therefore uses ρ to *prune* rather than to choose. A circuit with $\rho \ge 0.80$ is routed at $L_{\mathrm{tp}} \in \{4, 8, 16\}$ and every other circuit at $\{8, 16, 128\}$; the lowest-EPR route is kept. No instance above the threshold ever selected the long horizon, so pruning costs no EPR, and it removes the case where the long horizon fails to converge and consumes the whole attempt — the dominant compile-time term. Two rules keep the cost bounded and the result safe. The circuit's own default horizon is routed first and unguarded, and wins ties, so selection can never return a worse route than routing once; the remaining candidates then share a wall-clock budget proportional to what that first route cost, and one that overruns it is abandoned. The budget is what makes a deep horizon affordable to try, and it is also the one place PORTER's output depends on the machine it runs on (Section IV-C).

This rule was derived from the same corpus it is evaluated on, and we do not claim otherwise. The sweep above, the two candidate sets and the 0.80 threshold were all fixed by inspecting the 48 benchmark instances; there is no held-out family. Evidence for the *size* of the gain it delivers is therefore in-sample, and the pruning rule in particular should be read as a cost-saving heuristic fitted to this benchmark rather than as a validated predictor. Two things nonetheless bound the risk. The rule is a portfolio, not a prediction: the configured horizon is always a candidate and wins ties, so a badly chosen candidate set can cost compile time but cannot make the selected route worse than routing once. And the direction the sweep exposes — that no single constant serves circuits whose interaction densities differ by an order of magnitude — does not depend on the particular values chosen. An out-of-sample validation, by leave-one-family-out or on a fresh benchmark, is the natural next step and we have not performed it.

*2) Backward refinement (*PORTER+*R):* DSABRE routes each candidate layout with a forward → backward → forward schedule, so a protocol that matches its routing effort rather than only its layout count is needed to separate the two effects. PORTER+R supplies one. The schedule is SABRE's bidirectional layout refinement [12] applied to whole routes. Let $R(C, \ell)$ denote routing C from ℓ and ℓ' the layout it ends on. Pass 1 computes $R(C, \ell_0)$; pass 2 computes $R(C^{\mathrm{rev}}, \ell'_1)$ on the reversed circuit and *discards its route*, keeping only ℓ'_2; pass 3 computes $R(C, \ell'_2)$. Reversing C reverses every dependency edge, so routing it from the forward pass's end state yields a layout adapted to the *start* of the forward circuit. Odd pass counts above three repeat the cycle; $P{=}5$ throughout. Because the best forward pass is selected, PORTER+R can never report a higher EPR count than the pass 1 route it contains — at the cost of P routes where one would do. It adds no randomness.

One property of the schedule matters for reading its results. Refinement consumes a post-routing layout, and routing destroys the per-core free-slot reservation ℓ_0 was built with (Section III-A), so an intermediate pass may start from a layout worse than ℓ_0 for PORTER's own capacity machinery. Individual passes therefore regress on some instances, and only the best-pass rule keeps that from surfacing.

*3) Layout-discount search (*PORTER+*S):* PORTER+S sweeps the one free scalar in the layout heuristic — the per-layer discount γ of Section III-A, fixed at 0.99 elsewhere — over $\{0.90, 0.95, 0.99, 1.0\}$, routes each resulting layout, and keeps the lowest-EPR route. It is a search inside PORTER's own layout construction rather than an external partitioner or a seed sweep, and 0.99 is always among the candidates, so it too cannot return a worse route than the single pass.

PORTER+SR runs horizon selection and the γ-search and keeps the lower EPR of the two, netting the one route they share. The two look for different things — the horizon at a fixed layout against the layout at a fixed horizon — so neither subsumes the other.

TABLE I
EVERY CONFIGURATION THAT APPEARS IN SECTION V: LAYOUT SOURCE, LAYOUT COUNT $|\mathcal{L}|$, ROUTING PASSES PER LAYOUT P, TOTAL ROUTES $|\mathcal{L}|\cdot P$, AND THE QUANTITY MINIMISED OVER THEM. LAYOUT COUNTS MARKED ^{a} ARE 6 AT 25 AND 80 QUBITS AND 3 AT 64, WHERE DSABRE'S TWO CORNER RULES COINCIDE. LOWER EPR IS BETTER EVERYWHERE.

Name	Compiler	Layout source	$\lvert\mathcal{L}\rvert$	P	routes	selects on
Own protocol (RQ1, RQ4)						
TELESABRE	TELESABRE	Hungarian, seeds 0–2	3	1	3	best EPR
DSABRE	DSABRE	SabreLayout, seeds 0–2, both corner rules	6^a	3	9–18	best EPR
PORTER	PORTER	temporal locality-greedy	1	3	3	best EPR
PORTER+R	PORTER	temporal locality-greedy	1	5	5	best forward pass
PORTER+S	PORTER	$\gamma \in \{0.90, 0.95, 0.99, 1.0\}$	4	1–3	9–13	best EPR
PORTER+SR	PORTER	the above two, pooled	4	—	11–15	best EPR
Shared layout (RQ2) — both compilers routed from one external layout						
DSABRE shared	DSABRE	TELESABRE's or pytket-dqc's	1	3	3	best pass
PORTER shared	PORTER	the identical layout	1	3	3	best EPR
PORTER+R shared	PORTER	the identical layout	1	5	5	best forward pass
Cross layout (RQ2) — each compiler on the other's own layout						
DSABRE on PORTER	DSABRE	PORTER's layout	1	3	3	best pass
PORTER on DSABRE	PORTER	DSABRE's candidates	6^a	3	9–18	best EPR
Ablation (RQ3) — one fixed layout, one fixed horizon						
ablation legs	PORTER	temporal locality-greedy	1	1	1	—

G. Comparison Protocols and Budgets

Table I states, for every configuration that appears in the results, where its initial layouts come from, how many routing passes it spends, and what it selects on. We use the names in its first column throughout.

Three notions of a matched budget are in play and none is universally fair, so we report against more than one. Equal *route counts* (PORTER shared against DSABRE shared, three each) equalise how many times the router is invoked. Equal *layout budgets* put PORTER on DSABRE's candidate set. Equal *wall-clock* is not enforced anywhere; instead every table reports complete compile time, so a reader can renormalise. PORTER costs more per routing pass than DSABRE (Section V-F), so equal route counts favour PORTER on time and disfavour it on effort, and the reverse holds for equal layout budgets.

H. Metrics and Aggregation

The primary metric is EPR pairs consumed: TELESABRE counts teledata plus telegate operations, DSABRE counts teledata, and PORTER counts teleports plus telegates plus telegate packets, one pair per packet regardless of how many remote CX gates it amortises. All three are counts of the same physical resource. Intra-core SWAP counts and compile time are reported as secondary metrics and are never traded against EPR in any selection rule.

Aggregates are unweighted geometric means over circuits, $\left(\prod_i x_i\right)^{1/m}$, so each circuit contributes equally and a suite with large absolute counts does not dominate. Percentage changes are $100(b/a - 1)$ against the stated baseline a, and ratios b/a; both are computed from the geometric means, not averaged per circuit.

Failures are never imputed. A configuration that does not complete an instance is marked and *excluded from every geometric mean it appears in*, together with the instance's other columns where the comparison is stated as paired — so a mean labelled (n) is over exactly those n circuits all its columns completed. This keeps quality and completion separate, and it means a baseline that fails its hardest instances is compared only on the ones it survives, which flatters it. We therefore always report completion counts alongside, and Section V-A states both.

V. RESULTS

Each subsection answers one research question of Section IV-A. Protocol names and budgets are those of Table I; lower EPR is better throughout.

A. RQ1: End-to-End Cost Under Each Compiler's Own Protocol

Question: how do the three compilers compare when each uses its own recommended protocol? *Held fixed:* nothing — each compiler supplies its own layouts and pass schedule (Table I). *Metric:* geometric-mean EPR over jointly completed circuits, plus completion counts and complete compile time.

TELESABRE's EPR count is teledata plus telegate operations; DSABRE's is teledata only; PORTER's is teleport plus telegate plus telegate-packet operations (one EPR pair per packet, regardless of how many remote CX gates it amortises). All three report in the same one-EPR-pair unit. SWAP counts intra-core SWAPs only.

Against TELESABRE the ordering is consistent across the suite. From one deterministic layout PORTER lowers geometric-mean EPR by 59.6%, 41.6% and 68.2% at 25, 64 and 80 qubits, spending three routing passes against TELESABRE's three seeded attempts. Refinement adds to that at 25 and 64 qubits (63.6% and 47.6%) but not at 80, where the single selected pass is already the stronger of the two (68.2% against 63.2%) — the first place in this evaluation where choosing the lookahead horizon (Section IV-F1) beats spending four more routing passes. Every converged circuit improves, from −16.0% to −81.0%. PORTER completes all twenty-four instances, as does DSABRE; TELESABRE completes seventeen, failing on 64q Random and on six of the nine 80-qubit circuits including the 23,381-CX Random.

Two details are worth recording. On all eight 64-qubit circuits TELESABRE completes, our harness reproduces the EPR and SWAP counts DSABRE publishes for it [11] exactly, so the two evaluations share a baseline *measurement*, not merely a baseline tool. And the chain-structured GHZ that defeats locally-greedy schedulers, where TELESABRE spends 16 pairs on a 63-CX circuit, costs PORTER and DSABRE 6 each: PORTER's first-appearance visit order puts successive chain neighbours in one core, dissolving the pathology before routing begins.

Against DSABRE the answer depends on the pass budget and we report every point. At its own three-route protocol PORTER takes the geometric mean at 25 qubits (43.1 against 44.0) and trails at 64 and 80 (166.2 against 144.6; 38.5 against 36.5, both on the TELESABRE-matched basis), winning eight instances, losing twelve and tying four. Refinement comes within 3.2% at 64; the γ-search of Section IV-F3 closes the 64-qubit suite outright (172.7 against 173.1 over all nine circuits) and leaves 1.08× at 80. Where PORTER wins it wins large — 64q Multiplier is 763 against 1,285 with refinement, the widest margin here — and where it loses worst the gap is

TABLE II

MAIN COMPARISON, OWN PROTOCOL: EACH COMPILER FROM ITS OWN INITIAL LAYOUT. TELESABRE AND DSABRE REPORT THE BEST OF THREE SEEDS; PORTER ROUTES ONE DETERMINISTIC LAYOUT AT EACH OF THREE CANDIDATE LOOKAHEAD HORIZONS (SECTION IV-F1), AND PORTER+R SPENDS FIVE ROUTES ON THE BACKWARD-REFINEMENT SCHEDULE OF SECTION IV-F2 INSTEAD — CLOSER TO DSABRE'S NINE TO EIGHTEEN, THOUGH STILL SHORT OF THEM. RICHER SCHEDULES THAT DO REACH THAT BUDGET, AND PASS DSABRE, ARE IN TABLE VII. Δ_{TS} IS THE EPR CHANGE OF WHICHEVER PORTER COLUMN IS STRONGER ON THAT ROW, AGAINST TELESABRE (%); NEGATIVE IS BETTER. THE BOLDED PORTER COLUMN IS THAT STRONGER ONE — REFINEMENT LEADS AT 25 AND 64 QUBITS, HORIZON SELECTION ALONE AT 80. *TELESABRE DOES NOT CONVERGE IN THREE SEEDS; EACH GMEAN ROW COVERS THE CIRCUITS IT COMPLETES. LOCAL SWAP COUNTS, WHICH FOLLOW THE SAME ORDERING THROUGHOUT, ARE TABULATED IN APPENDIX H.

Suite	Circuit	TELESABRE	DSABRE	PORTER	PORTER+R	Δ_{TS}
25q	ae	119	65	65	65	−45.4
25q	ghz	13	4	4	4	−69.2
25q	graphstate	21	6	7	4	−81.0
25q	qft	138	81	74	74	−46.4
25q	qnn	515	143	119	116	−77.5
25q	random	636	404	401	383	−39.8
gmean (6)		106.6	44.0	43.1	**38.8**	**−63.6**
64q	ae	353	165	157	177	−55.5
64q	ghz	16	6	6	6	−62.5
64q	graphstate	62	15	20	16	−74.2
64q	qft	312	179	262	262	−16.0
64q	qnn	889	538	638	537	−39.6
64q	random	*	732	737	737	—
64q	qpeexact	289	187	222	222	−23.2
64q	qaoa	956	555	775	607	−36.5
64q	multiplier	1601	1285	1076	763	−52.3
gmean (8)		284.5	144.6	166.2	**149.2**	**−47.6**
80q	ae	*	332	299	490	—
80q	ghz	26	7	7	7	−73.1
80q	graphstate	87	20	28	24	−72.4
80q	qft	*	315	446	393	—
80q	qnn	*	791	1070	1181	—
80q	random	*	10551	11040	11040	—
80q	qpeexact	784	347	291	526	−62.9
80q	qaoa	*	862	1122	1122	—
80q	multiplier	*	3793	2411	2699	—
gmean (3)		121.0	36.5	**38.5**	44.5	**−68.2**

1.5×, on 80q QPEexact (526 against 347) and 64q QFT (262 against 179); the first of those is a loss for refinement alone, the selected route taking the instance outright at 291. The DSABRE revision measured here lowers its EPR on every one of our instances relative to its published tables, by up to 40% (Section IV-D). Against the earlier version a single PORTER pass led on all three suites; against this one it does not, and we report the current numbers.

B. RQ2: Routing Quality With the Layout Controlled

Question: with the initial layout held identical and the routing budget matched, does either routing loop use fewer EPR pairs? *Held fixed:* the initial layout, supplied externally. *Metric:* geometric-mean EPR, and win/loss counts per instance.

The own-protocol comparison mixes two effects: the quality of each compiler's initial layout and the quality of its routing loop. Table III removes the first — both DSABRE and PORTER route, single-shot, from the identical layout TELESABRE built for itself — leaving a pure routing comparison.

TABLE III

SHARED-LAYOUT COMPARISON: DSABRE AND PORTER BOTH ROUTED FROM THE IDENTICAL LAYOUT TELESABRE BUILT FOR ITSELF, SO THE ONLY DIFFERENCE LEFT IS THE ROUTING LOOP. GEOMETRIC-MEAN EPR OVER THE CIRCUITS TELESABRE COMPLETES AND THEREFORE PROVIDES A LAYOUT FOR — ALL SIX AT 25Q, EIGHT OF NINE AT 64Q, THREE OF NINE AT 80Q. Δ_{TS} COLUMNS ARE EACH COMPILER'S CHANGE AGAINST TELESABRE'S OWN-PROTOCOL RESULT. PER-CIRCUIT ROWS ARE IN APPENDIX H.

		DSABRE		PORTER	
Suite	TELESABRE	EPR	Δ_{TS}	+R	Δ_{TS}
25q (6)	106.6	51.0	−52.2	**47.7**	**−55.2**
64q (8)	284.5	189.2	−33.5	**157.2**	**−44.7**
80q (3)	121.0	74.2	−38.7	**55.7**	**−53.9**
all (17)	173.0	100.9	−41.7	**86.0**	**−50.3**

This experiment controls the initial layout, so it isolates the routing loop; it is also the one PORTER's own protocol loses. At three routes each DSABRE leads at 25 and 80 qubits and PORTER at 64 (−52.2% against −41.5%, −38.7% against −33.1%, −33.5% against −36.5%), pooling to 100.9 against 107.7 over the seventeen instances (the selected column is in Appendix H; Table III reports the refined one). Spending those three routes on lookahead horizons is therefore not what closes the gap; spending five on the forward–backward–forward schedule is, and by a clear margin — PORTER+R reaches 86.0 against DSABRE's 100.9 and leads at every scale. So it is that schedule, not the plan-committed loop, that separates the two compilers on fixed layouts; Section V-C confirms this from the other direction: removing plan commitment moves EPR by only 0.6%, and in PORTER's favour at that.

Pooled fixed-layout grid. A second source of externally supplied layouts — `pytket-dqc`'s hypergraph placements, routed by both compilers on the 25q and 64q suites (Appendix B; Appendix C offers that partition to PORTER as an extra layout candidate instead) — pools with Table III to give forty-one instances on which DSABRE and PORTER routed the *identical* layout. Under PORTER's own protocol DSABRE wins twenty to sixteen with five ties, pooled geometric mean 134.2 against 139.0; with the refinement schedule that matches its pass count PORTER wins twenty to fifteen with six ties, 126.6 against 134.2. Routing quality, with the layout controlled and the pass budget matched, does not separate these two compilers.

Each compiler on the other's layout. Closing the remaining cell — routing each system from its own and from its counterpart's layout (Appendix H) — gives the same reading from the other side. Down a column the compilers are close: on DSABRE's layout, DSABRE reaches 163.9 and PORTER+R 161.1; on PORTER's, 171.2 and 171.4. Across a row the layouts are not: DSABRE's candidate set is the better start for *both*, by 4.3% for its own router and 6.0% for PORTER's. The best cell is PORTER+R on DSABRE's layout.

At the sizes this grid covers, then, the routing stages are worth about the same given the same start, and what separates the two end-to-end is that DSABRE searches up to six candidate layouts where PORTER builds one — a deliberate trade (Section V-F) that Section IV-F3 prices. It does not hold

TABLE IV
MECHANISM ABLATION: PORTER EPR WITH ONE DESIGN DECISION DISABLED AT A TIME, ALL FROM THE SAME TEMPORAL LOCALITY-GREEDY LAYOUT. COLUMNS: "NO PK" DISABLES PACKETS (MAX_PACKET_SIZE=1), "NO REL." DISABLES BOTH RELIEF TIERS, "HOP-WISE" REPLACES PLAN COMMITMENT WITH PER-ITERATION RE-SCORING OF THE WHOLE FRONT, "TELEDATA" REMOVES THE TELEGATE FAMILY; "PK" AND "TG" ARE THE PACKET AND SINGLE-TELEGATE COUNTS OF THE FULL CONFIGURATION. THE LAST GMEAN ROW PAIRS FULL AGAINST HOP-WISE OVER ALL TWELVE ROWS, WHICH BOTH COMPLETE. †FAILS TO COMPLETE (DEADLOCK OR ITERATION-GUARD TRIP). ‡COMPLETES ONLY WITH THE ADDITIONAL RELIEF TIER DESCRIBED IN THE TEXT, WHICH THE TELEGATE OTHERWISE MAKES UNNECESSARY. THE FIRST GMEAN ROW IS OVER THE FOURTEEN INSTANCES THE FIRST THREE CONFIGURATIONS SHARE; THE SECOND IS THE TELEDATA-ONLY COLUMN PAIRED AGAINST FULL OVER THE SIXTEEN INSTANCES IT COMPLETES.

Suite	Circuit	Full	No pk	No rel.	Hop-wise	TELEDATA	pk	tg
25q	ae	65	71	80	65	68‡	8	0
25q	qft	74	74	140	74	74	0	0
25q	qnn	119	122	212	128	119	0	0
25q	random	401	412	395	399	413‡	43	93
64q	ae	177	211	327	172	198‡	32	0
64q	qft	262	372	382	336	**216**‡	45	1
64q	qnn	864	931	600	785	**694**‡	240	5
64q	random	737	843	789	805	†	77	228
80q	ae	490	688	†	383	**354**‡	264	15
80q	qft	446	435	†	415	**307**‡	209	2
80q	qnn	1517	1571	†	1474	**926**‡	448	30
80q	random	11040	11628	†	11280	†	572	3538
gmean (14)		57.6	61.4	67.0				
gmean (16)		78.3	83.9			**71.7**		
gmean (12)		413.3			411.7			

at every size: Section V-D shows the same control becoming counter-productive from 150 physical qubits upward, where a different parameter binds.

C. RQ3: Mechanism Ablation

Question: what does each mechanism contribute on its own? *Held fixed:* the initial layout, the lookahead horizon and every other parameter; exactly one mechanism is disabled per column. *Metric:* EPR, completion, and — for plan commitment, whose claim is about cost — compile time.

Table IV attributes PORTER's EPR consumption to its mechanisms by disabling each in isolation, holding the initial layout and every other parameter fixed; the per-action counts behind these rows are tabulated in Appendix G (the six omitted rows — GHZ and Graphstate at every scale — form no packets, never trigger relief and issue no telegates, so all configurations coincide there). Two columns go further than a per-primitive ablation: "Hop-wise" replaces PORTER's unit of commitment with DSABRE's while holding scoring, plan set and relief tiers fixed, and "TELEDATA" removes the TELEGATE family outright, leaving state teleportation as the sole inter-core action.

Packets. Disabling TELEGATEPACKET makes every shared-control remote CX run pay per gate, and the cost tracks the packet count: $+42\%$ EPR on 64q QFT (262 $\rightarrow$ 372, 45 packets), $+40\%$ on 80q AE (490 $\rightarrow$ 688, 264 packets), $+19\%$ on 64q AE, against a $+7\%$ overall gmean; the lone counter-move (80q QFT, 446 $\rightarrow$ 435) shows packet formation is a heuristic commitment that can displace a slightly better teleport plan. Read alone, this column is a clean win for the primitive. The next one overturns that reading.

TELEDATA only. Removing the TELEGATE family exposes two things the per-primitive ablations cannot. First, the TELEGATE is what makes PORTER *terminate*, not merely what makes it cheap: it is the only remote action needing no free destination slot, so it is always a legal plan for an adjacent remote CX. Without it a front whose remote CXs are all core-adjacent and whose destinations sit at or below $f_{\min}$ has an empty plan set, while the tail-relief BFS is only ever offered the *non-adjacent* blocked candidates — ten of the twelve rows deadlock on this. Feeding adjacent-blocked candidates into the same BFS (marked ‡, and measurably inert whenever the TELEGATE is available) recovers all but the two Random rows, which livelock — and those two are the most telegate-heavy in the suite (228 and 3,538 single telegates), so the primitive is load-bearing precisely where it is used most.

Second, where TELEDATA-only completes it is *cheaper than the full action set*, increasingly so with scale: $+2\%$ at 25q, mixed at 64q, and -33% gmean at 80q; overall 8.5% below Full on the sixteen instances it completes. The circuits it wins are exactly the packet-heaviest, making the ordering there TELEDATA only $<$ Full $<$ No packets: packets clearly beat the single telegates they degrade into, but on structured circuits at scale a pure teleportation policy beats both. We read this as myopia in the plan comparator rather than a verdict on packets — a packet's benefit is realised immediately and in full, a teleport's is latent and discounted across a bounded horizon — but we report it as measured. It also answers a question DSABRE states as open: its teledata-only vocabulary was chosen by measurement, an ungrouped telegate serving one gate where teledata amortises many, with grouping left as the open case. Grouping does close that per-EPR gap against single telegates — and still does not justify spending pairs that teleporting would have spent better.

Plan commitment. Replacing PORTER's unit of commitment with DSABRE's — returning to the top of the loop after one round of preparation SWAPs, so the front is re-classified and every plan re-scored before the next move — leaves EPR consumption statistically unchanged: 413.3 against 411.7 over the twelve rows here and 148.8 against 148.0 over the twenty-one instances we ran it on, with eight circuits better without commitment, five worse, eight tied, and all twenty-one completing either way. The two also make the *same number of routing moves* (128,222 against 128,565 iterations), which is the key number: commitment does not change what the compiler decides, only what each decision costs. What it buys is compile time — $1.31\times$ in geometric mean and $3.07\times$ on 80q Random — because a committed plan is not re-derived and re-scored every iteration. It is therefore an efficiency mechanism rather than a quality one: what makes a single PORTER pass cheap enough to be a competitive operating point, not what makes that pass good.

Capacity relief. Disabling both relief tiers is not merely more expensive — it is *unsafe*: 80q AE, QFT, QNN and Random all fail to complete, because with every candidate destination below $f_{\min}$ and no mechanism to drain qubits

TABLE V

SCALE-UP SUITE: QFT AND QPEEXACT AT 100, 200, AND 360 LOGICAL QUBITS, EPR PAIRS. ARCHITECTURES FOLLOW DSABRE'S OWN SCALABILITY PROTOCOL — 5×5 CORES THROUGHOUT, CORE COUNT $6 \rightarrow 12 \rightarrow 20$ — READ FROM THE BASELINES' OWN DEVICE FILES SO ALL ROUTERS SEE IDENTICAL PHYSICAL QUBIT IDS; OUR DSABRE COLUMN REPRODUCES THE FIGURES IT PUBLISHES FOR EXACTLY THIS SERIES [11] (201/246/766/896/1890/1938) TO WITHIN THE ONE CANDIDATE LAYOUT OUR PROTOCOL ADDS. THE LAST TWO COLUMNS ARE THE FIXED-LAYOUT CONTROL: PORTER ROUTING DSABRE'S OWN SABRELAYOUT CANDIDATES. "t" IS ONE DSABRE ATTEMPT (SECONDS); ITS PROTOCOL RUNS THREE PASSES OVER EACH OF UP TO SIX CANDIDATES. PYTKET-DQC IS OMITTED: ITS HYPERGRAPH PARTITIONER DOES NOT FINISH IN REASONABLE TIME AT THESE WIDTHS.

		architecture			DSABRE		PORTER, own layout		PORTER, DSABRE layout	
Circuit	CX	n	P	K	EPR	t	1 pass	+R	1 pass	+R
QFT	3,420	100	150	6	200	1.7	198	339	240	273
QPEexact	3,579	100	150	6	246	2.0	200	262	200	283
QFT	7,220	200	300	12	765	6.5	579	1128	1031	1018
QPEexact	7,579	200	300	12	896	7.8	528	979	651	946
QFT	13,300	360	500	20	1890	29.9	1156	2822	1870	2546
QPEexact	13,979	360	500	20	1938	38.9	1224	2342	1444	2416
gmean (6)					**705.7**		**507.7**	930.3	665.7	877.9

TABLE VI

ARCHITECTURE-INDEPENDENCE CHECK: THE NINE-CIRCUIT 64-QUBIT SUITE ON FOUR IBM 27-QUBIT HEAVY-HEX CORES (108 PHYSICAL QUBITS), WIRED AS A *ring* (4 LINKS, EVERY CORE DEGREE 2) AND A *star* (3 LINKS, CORE 0 A HUB EVERY ROUTE MUST CROSS). GEOMETRIC-MEAN EPR; THE RIGHT-HAND PAIR IS THE FIXED-LAYOUT CONTROL, PORTER ROUTING DSABRE'S OWN SABRELAYOUT CANDIDATES. PORTER COMPLETES ALL EIGHTEEN INSTANCES, SO EVERY ROW IS OVER THE FULL NINE; PORTER+R, WHICH DOES NOT SELECT A LOOKAHEAD HORIZON, STILL FAILS STAR MULTIPLIER AND IS THEREFORE "—" ON THAT ROW. PER-CIRCUIT ROWS ARE IN APPENDIX H. DSABRE REPRODUCES ITS PUBLISHED NUMBERS FOR THESE TWO ARCHITECTURES ON ALL EIGHTEEN INSTANCES [11].

		own layout		DSABRE layout	
Core graph	DSABRE	PORTER	+R	PORTER	+R
ring (9)	85.2	86.1	82.3	82.9	**81.4**
star (9)	101.1	122.3	—	98.5	**93.5**
star Multiplier	722	**455**	—	462	462

toward reservoirs the deterministic-greedy loop has no legal remote plan left. Where routing does complete, dense circuits pay heavily (+89% on 25q QFT, +78% on 25q QNN, +85% on 64q AE). The two instances where relief *hurts* — 64q QNN (864 → 600 without it) and, marginally, 25q Random (401 → 395) — show the opportunistic tier can overpay where congestion would have resolved itself. Of the four mechanisms this table ablates, capacity relief is the one whose removal costs completion rather than only EPR pairs: with both tiers enabled no instance in this evaluation reached the raise-on-deadlock path of Section III-H, and with both disabled four did. That is a statement about the instances evaluated here, not a robustness guarantee. (The initial layout, which this table holds fixed rather than ablating, is assessed separately — Sections V-B and V-E.)

D. RQ4: Scale-Up on QFT and QPEexact to 360 Logical Qubits

Question: do the RQ1 and RQ2 answers hold as circuit width and core count grow together? *Held fixed:* DSABRE's own scalability architectures and circuit files. *Metric:* geometric-mean EPR and complete compile time.

Every result so far is at 108 physical qubits or fewer. Table V takes the two circuit families that exist at 100, 200 and 360 logical qubits onto DSABRE's own scalability series: 5×5 cores throughout with the core count growing $6 \rightarrow 12 \rightarrow 20$, so occupancy is roughly constant while the core graph grows. It is a monotone ladder in the quantity that matters: inter-core traffic scales with core count, and DSABRE's own cost rises $200 \rightarrow 765 \rightarrow 1{,}890$ on QFT across it.

On these two families PORTER uses fewer EPR pairs than DSABRE on all six instances, by a widening margin: 0.90× DSABRE's geometric-mean EPR at 100 logical qubits, 0.67× at 200 and 0.62× at 360, for 0.72× overall. It does so at 0.20× DSABRE's complete compile cost (8.4 s against 41.1 s in geometric mean), because DSABRE routes up to eighteen passes here against PORTER's three.

What carries that result is the lookahead horizon, and nothing else does. Both mechanisms that close PORTER's gap below 108 physical qubits are inert or harmful at these widths. Backward refinement loses on all six instances, by 83% in geometric mean. Handing PORTER DSABRE's own `SabreLayout` candidates — the fixed-layout control that reaches parity at 25 and 64 qubits (Section V-B) and *wins* on both heavy-hex core graphs (Section V-E) — makes it *worse* here, 0.94× against 0.72×, and worst on the largest instance, where DSABRE's layout costs 1,870 pairs against 1,156 from PORTER's own. The mapping is not the binding constraint at this scale. The horizon is: every instance in this series selects L_{tp}=128, the longest candidate, and at the shipped 16 PORTER instead trails DSABRE by 1.45× (Section IV-F1).

This separates two deficits the smaller suites conflate. Up to 108 physical qubits what limits PORTER is its initial layout: control it and match the pass budget and the difference between the two compilers disappears. From 150 upward the layout is no longer the constraint and the horizon is — consistent with its being an absolute gate count that does not grow with K, so that at twenty cores a plan is scored over proportionally far less of the circuit than at four. Selecting it per circuit is a workaround rather than a fix, and a K-aware horizon remains open.

E. RQ4: Heavy-Hex Cores and Non-Grid Core Graphs

Question: how much of the foregoing depends on square cores wired in a grid? *Held fixed:* the 64-qubit circuit suite and the physical-qubit count; the core shape and core graph change. *Metric:* geometric-mean EPR and win/loss counts.

Every result so far uses square grid cores wired in a grid. To test how much of PORTER's behaviour depends on that we adopt DSABRE's own non-grid experiment [11]: the same nine-circuit 64-qubit suite on four IBM 27-qubit heavy-hex tiles. Two things change at once. The *core* becomes irregular — degrees over $\{1, 2, 3\}$, six degree-1 leaves, diameter 12 against a 4×4 grid's 6 — and the *core graph* is either a ring, where every core has degree 2, or a star, where core 0 is a hub and the others are leaves. PORTER's hardware

object is derived from the architecture DSABRE's own builder produces, and a topology assertion checks both compilers see identical qubit ids, intra-core edges and port endpoints before anything is routed; DSABRE routes on its native object, reproducing its published numbers on all eighteen instances. Nothing in either compiler is specialised for a heavy-hex tile.

The core graph matters; the core shape does not. Table VI shows an irregular tile costs PORTER nothing: on the ring PORTER+R *leads* DSABRE (82.3 against 85.2), winning six of nine and tying GHZ — better relative standing than it holds on the grid suite of Table II. Doubling the intra-core diameter and replacing degree-4 grid interiors with degree-1 leaves disturbs neither compiler. The star does: under its own protocol PORTER trails there, 122.3 against 101.1, winning two of nine, and PORTER+R — which cannot route Multiplier at all — trails by 13.6% on the eight it completes.

Star Multiplier is a livelock the horizon escapes. At a fixed L_{tp}=16 this instance does not terminate, and it fails as a local-SWAP livelock rather than the capacity deadlock of Section III-H: retired gates freeze at 3,226 of 13,040 and the EPR count at 538 while intra-core SWAPs accumulate at one per iteration, so the iteration guard rather than the progress argument stops it. It is a property of the horizon, not of the circuit: the same layout and the same router complete it at L_{tp}=128, at 455 pairs against DSABRE's 722. Since horizon selection routes that candidate as a matter of course, PORTER has no unrouted instance anywhere in this evaluation — whereas PORTER+R, which holds the horizon fixed, still fails it.

The star gap is the layout, not the routing. Given DSABRE's own layout PORTER completes all nine star circuits — including Multiplier at 462 pairs against DSABRE's 722, which its own layout now also routes, slightly better, at 455 — and with the pass budget matched the control reverses the result on *both* core graphs (-4.4% on the ring, -7.5% on the star), the larger margin falling on the topology PORTER loses outright under its own protocol. The cause is visible in the layout: `build_initial_layout` reserves $f_{\min}$ in *every* core, treating cores as interchangeable. They are on a grid and on a ring, where every core has the same degree; they are not on a star, where Multiplier opens with free counts $(2, 2, 13, 27)$ — the hub every route must cross pinned at the floor while a whole core sits empty. Raising the reservation uniformly does not repair it (we tried $f_{\min}\in\{4, 6\}$); what the star asks for is a reservation scaled by a core's position in the core graph, which we have not implemented.

Taken with Section V-D this bounds the layout reading rather than extending it. Up to 108 physical qubits — across two core shapes and three core graphs — the initial layout is what separates the two compilers, and controlling it erases the difference. Above 150 the layout stops being the constraint, and Section IV-F1's parameter takes over.

F. The EPR/Time Operating Frontier

PORTER's protocol routes one deterministic layout at each of three candidate lookahead horizons and keeps the best (Section IV-F1), and two further schedules spend more routes:

TABLE VII
PROTOCOL TRADE-OFF OVER ALL 24 INSTANCES. "ROUTES" IS THE NUMBER OF ROUTING PASSES; TIMES ARE COMPLETE COMPILE COSTS (GEOMETRIC MEAN PER INSTANCE). PORTER ROUTES EACH OF THREE CANDIDATE LOOKAHEAD HORIZONS AND KEEPS THE BEST (SECTION IV-F1); PORTER+R ADDS BACKWARD REFINEMENT (SECTION IV-F2) AT A FIXED HORIZON; PORTER+S ADDS THE BEST-OF-γ LAYOUT SEARCH OF THIS SECTION ON TOP OF IT; PORTER+SR TAKES THE BETTER OF HORIZON SELECTION AND THAT SEARCH, NETTING THE ONE ROUTE THEY SHARE. ALL ARE SEED-FREE.

Protocol	routes	EPR	vs. DSABRE	time (s)	vs. DSABRE
DSABRE	9–18	163.9	1.00	2.26	1.00
PORTER	3	174.9	1.07	1.32	**0.58**
PORTER+R	5	171.4	1.05	1.67	0.74
PORTER+S	9–13	163.2	1.00	4.59	2.03
PORTER+SR	11–15	**158.2**	**0.97**	5.65	2.50

PORTER+R adds the backward refinement of Section IV-F2, and PORTER+S additionally sweeps the one free scalar in the layout heuristic — the per-layer discount γ, pinned at 0.99 elsewhere — over $\{0.90, 0.95, 0.99, 1.0\}$ and keeps the best route by EPR. Both remain deterministic and seed-free, and the sweep is a search inside PORTER's own design rather than an external partitioner or a seed sweep. A fourth point falls out of the first three. Selection and the γ-search look for different things — the horizon at a fixed layout, against the layout at a fixed horizon — and neither dominates: the search gives the better route on fourteen of the twenty-four instances, selection on four, and they tie on six. PORTER+SR runs both and keeps the lower EPR, netting the one route they share. Table VII places the four points against DSABRE, pooled over all twenty-four.

The trade is time against EPR pairs, and it is monotone in both. PORTER costs $1.07\times$ DSABRE's EPR at $0.58\times$ its compile time; refinement reaches $1.05\times$ at $0.74\times$; the layout search reaches parity (163.2 against 163.9) at $2.03\times$; and PORTER+SR passes it, at $0.97\times$ its EPR for $2.50\times$ its compile time. Nothing here is free, because PORTER is dearer per routing pass and the richer schedules run more of them — 12.4 on average for the γ-search alone.

PORTER+SR leads DSABRE on twelve of the twenty-four instances against nine behind, and by construction never loses to either schedule it is built from. Increasing PORTER's deterministic search budget therefore closes and reverses the own-protocol gap. We stop short of concluding that the gap *is* a search-budget gap: the two schedules PORTER+SR adds change the initial layout as well as the route, and the comparison equalises route counts rather than wall-clock time, so this does not isolate search effort from the other things it varies. Where on the curve to sit depends on whether EPR pairs or compile seconds are scarce.

Selecting the horizon has largely overtaken refinement as an *operating point*: over the twenty-four instances PORTER+R is ahead by only 2.0% in geometric mean for four extra routes, and it is 12.0% *behind* the selected pass at 80 qubits, 83% behind on the scale-up series, and the configuration that fails star Multiplier. We report it throughout only because the fixed-layout comparison of Section V-B needs a PORTER schedule that spends its extra effort on routing passes rather than on

layouts, which PORTER+S by construction cannot be.

All times are *complete* compile costs, following DSABRE's own convention: layout generation plus every routing pass over every candidate, not the winning attempt alone. For DSABRE that is nine to eighteen passes, against three for PORTER and five for PORTER+R; TELESABRE's figure covers all three seeds. PORTER's advantage is therefore in the count of passes, not in their price: per routing pass it is the dearer of the two on all nine 64-qubit circuits, at $0.27\times$ DSABRE's per-pass speed in geometric mean, partly because enumerating whole plans, collecting packets by forward simulation and running a two-tier capacity search all cost more per iteration than scoring a flat list of hops, and partly because a deep horizon makes each iteration dearer still. On that nine-circuit subset the complete costs are 1.98 s against 1.86 s and DSABRE is the cheaper; pooled over all twenty-four PORTER is ahead at $0.58\times$.

Over the five circuits TELESABRE completes (per-circuit times in Appendix H), geometric-mean complete cost is 5.95 s for TELESABRE, 2.67 s for DSABRE, 1.55 s for PORTER and 1.97 s with refinement, and neither PORTER point holds that margin on the largest circuit here, where PORTER+R costs 191 s against 144 s at 23,381 CX. The cheap operating point is still PORTER, and at $1.07\times$ DSABRE's EPR it no longer gives up much to be so.

VI. Discussion and Limitations

Section V reports several results that are individually clear but that read as tensions when placed side by side. We resolve them here, then state what the evaluation does not establish.

What the plan abstraction buys, and what commitment buys. These are separable and the paper's own terminology has to keep them apart. The *plan-oriented* design — generating whole inter-core actions as candidates and ranking them on gain with preparation cost demoted to a tie-break — is what distinguishes PORTER's remote stage, and it is the configuration that produces the EPR reductions of Section V-A. *Commitment* is the narrower rule that the selected plan is not reopened until it is ready. Only the latter is ablated in Table IV: the hop-wise column re-derives and re-scores the same plan set every iteration, so it changes when the choice is revisited and nothing else. That it reaches the same EPR on the same number of routing moves says commitment is not where the quality comes from; it does not say the plan abstraction is idle, because no column removes it. Isolating the abstraction would need a hop-granular candidate set, which is a different router and which we have not built. This is the clearest gap in our attribution.

Layout below about a hundred physical qubits, horizon above it. PORTER trails DSABRE under RQ2 at ≤ 108 physical qubits, and controlling the layout while matching the pass budget erases the difference (Sections V-B and V-E). On the scale-up series neither the layout control nor extra refinement passes help — DSABRE's layout is worse for PORTER there than its own — while selecting a longer lookahead horizon moves PORTER from $1.45\times$ behind to $0.72\times$ ahead. The two regimes are consistent with the horizon being an absolute gate count that does not grow with the core count K, so that a plan at twenty cores is scored over proportionally less of the circuit than at four. We have not tested a K-scaled horizon, so that remains a hypothesis rather than a finding.

Why TELEDATA-only can beat the full action set. Packets clearly beat the single telegates they degrade into, yet removing gate teleportation entirely is cheaper still on the packet-heaviest circuits at 80 qubits. We read this as an interaction between the action vocabulary and the scorer rather than as a failure of packetisation. The two are scored in different currencies: a packet's gain is a count of remote CX gates it resolves immediately, whereas a TELEDATA plan's is a sum of core-distance reductions, most of it discounted over a bounded horizon. Nothing establishes that one resolved gate and one unit of distance reduction are commensurate, and where a teleport's repayment extends past the horizon the comparator cannot see it. The ablation is evidence that this mismatch costs EPR pairs at scale; it is not evidence that a pair spread over a run of gates is a bad idea.

What the completion record does and does not show. Section III-H separates correctness and termination, which PORTER guarantees by construction, from completion, which it does not. Every instance in this evaluation completes under the selected-horizon protocol, but that is a property of the protocol and the benchmark rather than of the router: at a fixed horizon, heavy-hex star Multiplier livelocks, and the extra candidate is a way of not meeting the failure rather than of excluding it. Two of the four failure modes were observed here — livelock at a fixed horizon, and infeasibility on an external pytket-dqc layout that fills a core — while capacity deadlock was observed only with relief disabled. DSABRE's safe mode carries a termination-with-success proof under stated architectural hypotheses, and supplying a comparable guarantee is the clearest piece of theory this design leaves open.

A. Threats to Validity

Benchmark coverage. The main suites are nine MQT-Bench families at three widths; the scale-up series is two families (QFT and QPEexact) only, so nothing here supports a claim about scaling behaviour on irregular or randomly structured circuits at those widths. The two heavy-hex networks are two core graphs, not a survey of architectures.

Parameter rules fitted in-sample. The horizon candidate sets and the density threshold were chosen by inspecting these same instances (Section IV-F1). The magnitude of the horizon-selection gain is therefore in-sample. The portfolio construction bounds the downside — the configured horizon is always a candidate and wins ties — but an out-of-sample validation has not been performed. The same caution applies in weaker form to $f_{\min}$, m_{assist} and the packet window, which were fixed early and not swept.

Reconstructed benchmark instances. The 80-qubit AE and QNN circuits are not produced by the stock generator and were reconstructed (Appendix A). We release the construction scripts and the files, and validate the QNN reconstruction by

exact CNOT-sequence match against the 64-qubit file, but they remain our circuits rather than the generator's.

Patched baseline binary. All TELESABRE numbers use a binary patched to avoid an out-of-range index in its initial-layout pass (Section IV-D). The patch changes placement only for qubits the stock binary leaves unplaced, and architecture-H results, where the stock binary does not crash, are unchanged by it; we have not otherwise audited its effect.

An evolving concurrent baseline. DSABRE was revised during this work. We pin an exact revision and re-run rather than quoting, but a later revision may move the comparison again.

Unequal native budgets. The own-protocol comparison gives DSABRE nine to eighteen routes against PORTER's three. This is each compiler's own convention rather than a controlled budget, which is why RQ2 exists and why compile time is reported everywhere.

Aggregation over failures. Geometric means are taken over jointly completed instances, so a baseline that fails hard instances is scored only on easier ones. Completion counts are reported alongside every such mean, but the two must be read together.

Implementation and timing. PORTER is Python and the baselines are Python and C respectively, so per-pass times compare implementations as much as algorithms. Compile times are wall-clock on one machine. Horizon selection additionally guards its speculative routes with a wall-clock budget, so on a loaded machine it may abandon a candidate that would otherwise have won: the protocol's output is reproducible only up to that budget, though each individual route is exactly reproducible.

Cost model. One EPR pair per inter-core action, independent of hop count and generation protocol; link contention, entanglement latency and fidelity, purification, and classical communication cost are all outside the model (Section II-B). Conclusions are relative to that model.

VII. Related Work

TELESABRE [10] is the published state of the art for this cost model: it supports teledata and gate teleportation and builds on SABRE's extended-set scoring. The two differ chiefly in decision structure. TELESABRE scores inter-core candidates through a contracted-communication-graph Dijkstra energy that folds staging cost and lookahead into one value, and — uniquely among the compilers here — places those candidates in the *same* pool as local SWAPs, taking the global minimum each iteration. PORTER keeps local work strictly ahead of inter-core work, scores whole plans independently of preparation cost, generates preparation SWAPs only for the plan it commits to, and adds the packet primitive. Its capacity handling is also more proactive: alongside TELESABRE's safety-valve cap it maintains an explicit blocked-plan set with a tail-relief search and a pre-emptive relief comparator.

DSABRE [11] is concurrent work on the same problem, and the two compilers are close relatives developed in parallel. Its router was substantially revised while this paper was in preparation — a shared extended-set construction and a safe-mode capacity discipline — lowering its EPR on every one of our own-protocol instances relative to its published tables, by up to 40%, and cutting its compile time by up to $97\times$. We therefore re-ran it from source throughout rather than quoting figures its current code does not reproduce, and our numbers match those it now publishes for the architectures it reports.

Stating what the two share keeps either from being credited with the other's ideas: both rank moves by SABRE's front-plus-lookahead Δ score; both are strictly locality-first, scoring an inter-core candidate only once no front gate is intra-core; both promote capacity from a scoring penalty to a legality condition with the same floor of two free slots; and both manufacture capacity by relaying a free slot from a reservoir along a BFS path rather than evicting arbitrarily. Four differences remain, and they are what this paper evaluates: PORTER's action vocabulary is larger (DSABRE ships teledata only); its unit of commitment is a whole plan rather than one hop; preparation cost enters as a tiebreaker rather than additively; and its initial layout is one deterministic construction rather than a seeded best-of-N search. DSABRE's safe mode additionally carries a completion guarantee that PORTER does not match (Sections III-H and VI).

TABLE VIII
DESIGN AXES OF THE COMPILERS DISCUSSED HERE. "COMMITMENT" IS THE UNIT A SCORING DECISION IS TAKEN OVER. `PYTKET-DQC` IS PARTITION-FIRST AND DOES NOT ROUTE WITH THESE PRIMITIVES, SO SEVERAL AXES DO NOT APPLY.

	TELESABRE	DSABRE	pytket-dqc	PORTER
Paradigm	online	online	partition-first	online
Primitives	data, gate	data	(e-bit)	data, gate, packet
Local/remote	one pool	staged	—	staged
Commitment	one move	one hop	whole plan	whole plan
Prep. cost	in score	in score	—	tie-break
Layout	Hungarian, 3 seeds	SabreLayout, 3 seeds	hypergraph partition	one deterministic
Capacity	cap	legality + relay	partition	legality + relief
Termination	—	with-success proof	—	correct, terminates
Metric	EPR	EPR	e-bits	EPR

Table VIII places the four systems on the axes that matter here.

Other approaches. Partition-first compilers assign qubits to cores by hypergraph or community partitioning [13]–[15] and route the residue; `pytket-dqc` [14] is the mature representative, compared against in Appendix B under the caveat that its e-bit unit assumes long-lived entanglement and different communication memory. AutoComm [8] identifies gate sequences that can share one entangled resource, which PORTER's packet applies inside a routing loop rather than as a pre-pass (Section II-C). SABRE [12] supplies the scoring skeleton all three compilers inherit.

VIII. Conclusion

PORTER is a deterministic DQC compiler built on a plan-oriented remote stage: it scores a complete inter-core action on the gate progress it buys, before any preparation SWAP for it exists, and commits before preparing. Around that it places a capacity discipline with two relief tiers, a seed-free initial layout, and a per-circuit choice of teleport-lookahead horizon.

Three conclusions are supported. First, on the evaluated MQT-Bench suites PORTER uses substantially fewer EPR pairs than published TELESABRE — 59.6%, 41.6% and 68.2% less in geometric mean at 25, 64 and 80 logical qubits, at a matched attempt budget — and completes every instance in all three suites, including the seven on which TELESABRE does not converge. Second, against the concurrent DSABRE revision measured here, PORTER is competitive rather than dominant, and which compiler leads depends on the operating point: it uses 7% more pooled EPR at 0.58× the compile time under each compiler's own protocol, and leads on identical externally supplied layouts, on both heavy-hex core graphs, and on the 100–360-qubit scale-up series. Third, the ablations attribute those results rather than leaving them to the system as a whole: capacity relief is what makes a deterministic-greedy design complete in practice, the initial layout is what separates the two compilers below roughly a hundred physical qubits, and the lookahead horizon is what separates them above it.

Two ablation results are negative and they delimit the contribution. Plan commitment does not lower EPR consumption — it makes the same number of routing moves as re-scoring the front each iteration and lands within 0.6% — so it is a compile-time mechanism, and no experiment here isolates the plan abstraction it rests on. And a richer gate-teleportation vocabulary does not always pay: at 80 qubits, removing gate teleportation entirely beats the full action set by 32.8% on the packet-heaviest circuits, which we read as the plan scorer undervaluing teleportation's latent return rather than as a verdict on amortisation.

The evaluation is bounded in ways worth restating: two circuit families at the largest widths, two non-grid core graphs, horizon-selection rules fitted on the same corpus they are measured on, and a cost model that charges one pair per inter-core action and ignores link contention, latency and fidelity (Section VI-A). The openings that follow are correspondingly concrete: a completion guarantee to match DSABRE's, an out-of-sample validation of the horizon rule, a lookahead horizon that scales with the core count rather than being selected by trial, a plan comparator that weighs a resolved gate against a latent distance reduction on a common scale, and a richer hardware cost model under which EPR count is no longer the whole objective.

ACKNOWLEDGMENTS

This work was partially supported by the Natural Science Foundation of China (Grant No. 12471437). The authors used Anthropic's Claude (via Claude Code) to assist with: (i) reviewing and applying behaviour-preserving performance fixes to the PORTER implementation, (ii) authoring the independent replay verifier and the cross-router comparison harness, (iii) diagnosing and patching the TELESABRE reference-implementation issue described in Section IV-D, and (iv) drafting the LaTeX source of this manuscript. All AI-generated content was reviewed and verified by the authors, who take full responsibility for the correctness and originality of the work.

REFERENCES

[1] D. Cuomo, M. Caleffi, and A. S. Cacciapuoti, "Towards a distributed quantum computing ecosystem," *IET Quantum Communication*, vol. 1, no. 1, pp. 3–8, 2020.

[2] P. Escofet, A. Ovide, M. Bandic, L. Prielinger, C. G. Almudever, S. Feld, E. Alarcón, and S. Abadal, "Revisiting the mapping of quantum circuits: Entering the multi-core era," *ACM Transactions on Quantum Computing*, 2024, arXiv:2403.17205.

[3] L. J. Stephenson, D. P. Nadlinger, B. C. Nichol, S. An, P. Drmota, T. G. Ballance, K. Thirumalai, J. F. Goodwin, D. M. Lucas, and C. J. Ballance, "High-rate, high-fidelity entanglement of qubits across an elementary quantum network," *Physical Review Letters*, vol. 124, no. 11, p. 110501, 2020.

[4] S. Daiss, S. Langenfeld, S. Welte, E. Distante, P. Thomas, L. Hartung, O. Morin, and G. Rempe, "A quantum-logic gate between distant quantum-network modules," *Science*, vol. 371, no. 6529, pp. 614–617, 2021.

[5] C. H. Bennett, G. Brassard, C. Crépeau, R. Jozsa, A. Peres, and W. K. Wootters, "Teleporting an unknown quantum state via dual classical and Einstein-Podolsky-Rosen channels," *Physical Review Letters*, vol. 70, no. 13, pp. 1895–1899, 1993.

[6] J. Eisert, K. Jacobs, P. Papadopoulos, and M. B. Plenio, "Optimal local implementation of nonlocal quantum gates," *Physical Review A*, vol. 62, no. 5, p. 052317, 2000.

[7] A. Yimsiriwattana and S. J. Lomonaco Jr., "Generalized GHZ states and distributed quantum computing," *arXiv preprint quant-ph/0402148*, 2004.

[8] A. Wu, H. Zhang, G. Li, A. Shabani, Y. Xie, and Y. Ding, "AutoComm: A framework for enabling efficient communication in distributed quantum programs," in *Proceedings of the 55th IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2022, arXiv:2207.11674.

[9] N. Quetschlich, L. Burgholzer, and R. Wille, "MQT Bench: Benchmarking software and design automation tools for quantum computing," *Quantum*, vol. 7, p. 1062, 2023.

[10] E. Russo, E. Vinciguerra, M. Palesi, D. Patti, G. Ascia, and V. Catania, "TeleSABRE: Heuristic layout synthesis in multi-core quantum systems with teleport interconnect," in *Proceedings of the IEEE International Conference on Quantum Computing and Engineering (QCE)*, 2025, pp. 749–758, arXiv:2505.08928. Code: https://github.com/Haimrich/telesabre.

[11] S. Li, "dSABRE: A SABRE-style router for multi-core distributed quantum computers," 2026, manuscript, Centre for Quantum Software and Information, University of Technology Sydney. Code: https://github.com/ebony72/dsabre.

[12] G. Li, Y. Ding, and Y. Xie, "Tackling the qubit mapping problem for NISQ-era quantum devices," in *Proceedings of the 24th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2019, pp. 1001–1014.

[13] P. Escofet, M. Bandic, C. G. Almudever, and S. Feld, "HQA: A hungarian-based qubit assignment for multi-core quantum computers," in *Proceedings of the IEEE Quantum Software Conference*, 2023.

[14] P. Andres-Martinez, D. Mills, T. Forrer, and L. Henaut, "CQCL/pytket-dqc," https://github.com/CQCL/pytket-dqc, Jun. 2024.

[15] O. Daei, K. Navi, and M. Zomorodi-Moghadam, "Optimized quantum circuit partitioning," in *International Journal of Theoretical Physics*, vol. 59, 2020, pp. 3804–3820.



Sanjiang Li received his B.Sc. in mathematics from Shaanxi Normal University in 1996 and his Ph.D. in mathematics from Sichuan University in 2001. He is currently a professor at the Centre for Quantum Software and Information at the University of Technology Sydney (UTS). Prior to joining UTS, he worked in the Department of Computer Science and Technology at Tsinghua University from 2001 to 2008. His primary research interests include knowledge representation, artificial intelligence, and quantum circuit compilation.